Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ
Предмет: Разработка интернет приложений
Лабораторная работа №4
“ Разработка серверной части и локального HTTP API”

Студент: Сигорский Александр Евгеньевич
Группа: ИБМ3-54Б
Преподаватель: М.В. Бурмистрова

Москва 2026

1. Описание задания

Цель лабораторной работы — рассмотреть создание серверной части веб-приложения: приём HTTP-запросов, маршрутизацию, формирование ответов, передачу данных в формате JSON и обработку ошибок.

В CodeSlicer эти требования реализованы внутри локального HTTP API. Сервер написан на Python и обслуживает тот же браузерный интерфейс, который рассматривается в лабораторной работе №3. Система работает local-first: основной анализ выполняется на компьютере пользователя, а результаты передаются браузеру через локальные HTTP-запросы.

CodeSlicer строит граф программного проекта и позволяет выполнять review изменений, исследовать связи, получать проекции графа и состояние анализа. В лабораторной работе серверная часть рассматривается как реальный backend этого приложения, а не как отдельный демонстрационный сервер.

2. Структура серверной части

Основные элементы backend, относящиеся к лабораторной работе:
• src/impact_engine/local_api.py — HTTP-обработчик и маршруты API;
• src/impact_engine/local_api_state.py — состояние выбранного проекта и анализа;
• src/impact_engine/local_api_domain.py — прикладные операции локального API;
• frontend/api-client.js — клиент, который использует API из браузера.

Сервер основан на стандартном HTTP-механизме Python. Класс LocalApiHandler наследуется от SimpleHTTPRequestHandler и реализует методы GET, POST и OPTIONS.

3. Текст программы и пояснения

3.1. Формирование JSON-ответа

Листинг 1 — метод отправки ответа клиенту:

```
class LocalApiHandler(SimpleHTTPRequestHandler):
    server_version = "ImpactEngineLocalAPI/0.5"

    def _send_json(self, status: int, payload: Any) -> None:
        body = json.dumps(
            payload,
            ensure_ascii=False
        ).encode("utf-8")

        self.send_response(status)
        self.send_header(
            "Content-Type",
            "application/json; charset=utf-8"
        )
        self.send_header(
            "Content-Length",
            str(len(body))
        )
        self.send_header(
            "Cache-Control",
            "no-store"
        )
        self.end_headers()
        self.wfile.write(body)
```

Метод принимает HTTP-статус и Python-объект payload. Затем объект сериализуется в JSON и кодируется в UTF-8. Заголовок Content-Type сообщает браузеру формат ответа. Cache-Control: no-store запрещает использовать устаревший кэш для динамического состояния проекта.

3.2. Чтение JSON из POST-запроса

Листинг 2:

```
def _read_json(self) -> dict[str, Any]:
    length = int(
```

```
        self.headers.get(
            "Content-Length",
            "0"
        )
    )

    if length > 2_000_000:
        raise ValueError(
            "Request body exceeds 2 MB"
        )

    value = json.loads(
        self.rfile.read(length) or b"{}"
    )

    if not isinstance(value, dict):
        raise ValueError(
            "Request body must be a JSON object"
        )

    return value
```

Сервер получает размер тела запроса из Content-Length и ограничивает его двумя мегабайтами. После чтения данные преобразуются из JSON в словарь Python. Если клиент передал массив, строку или другое значение вместо объекта, сервер возвращает ошибку. Такая проверка делает контракт API предсказуемым.

3.3. Ограничение локального доступа

CodeSlicer проектируется как local-first приложение. Поэтому backend дополнительно проверяет Host.

Листинг 3:

```
def _host_allowed(self) -> bool:
    host = (
```

```
        urlsplit(
            "//" +
            self.headers.get("Host", "")
        ).hostname or ""
    ).lower()

    if self.state.docker_local_ui        or not self.state.allow_remote:
        allowed = {
            "localhost",
            "127.0.0.1",
            "::1"
        }
    else:
        allowed = self.state.allowed_hosts

    if host in allowed:
        return True

    self._send_json(
        403,
        {
            "status": "error",
            "error": "local_host_required"
        }
    )
    return False
```

Проверка необходима не только для API, но и для статических ресурсов. В обычном режиме разрешены loopback-адреса localhost, 127.0.0.1 и ::1. Это соответствует модели, в которой исходный код пользователя анализируется локально.

3.4. GET-маршруты

Листинг 4 — часть метода do_GET:

```
if parsed.path == "/api/health":
```

```
    return self._send_json(200, {
        "status": "ok",
        "service":
            "impact-engine-local-api",
        "api_contract_version":
            LOCAL_API_CONTRACT_VERSION,
        "capabilities": {
            "managed_tools": True,
            "tools_endpoint": "/api/tools",
        },
        "session_token":
            self.state.session_token,
    })

if parsed.path == "/api/state":
    return self._send_json(
        200,
        self.state.snapshot(
            include_graph=False
        )
    )

if parsed.path == "/api/progress":
    return self._send_json(
        200,
        {
            "status": "ok",
            "progress":
                self.state.progress
        }
    )

if parsed.path == "/api/graph":
    snapshot =
        self.state.snapshot()

    if not snapshot.get("graph"):
```

```
        return self._send_json(
            404,
            {
                "error": "no_analysis",
                "message":
                    "Analyze a project first"
            }
        )

    return self._send_json(
        200,
        {
            "status": "ok",
            "project_path":
                snapshot["project_path"],
            "graph":
                snapshot["graph"]
        }
    )
```

Маршрут /api/health используется для проверки доступности сервера. /api/state возвращает текущее состояние приложения, /api/progress — прогресс длительного анализа, /api/graph — готовый граф. Для графа предусмотрена отдельная ошибка 404, если анализ ещё не выполнялся.

3.5. Запуск анализа через POST

Листинг 5:

```
if parsed.path == "/api/analyze":
    project_path = str(
        body.get("project_path")
        or self.state.default_project
        or ""
    ).strip()

    if not project_path:
```

```
            return self._send_json(
                400,
                {
                    "status": "error",
                    "error":
                        "project_path is required"
                }
            )

        try:
            return self._send_json(
                200,
                self.state.analyze(project_path)
            )
        except AnalysisCancelled:
            return self._send_json(
                409,
                {
                    "status": "cancelled",
                    "progress":
                        self.state.progress
                }
            )
        except Exception as exc:
            with self.state.lock:
                self.state.last_error = str(exc)

            return self._send_json(
                422,
                {
                    "status": "error",
                    "error": str(exc)
                }
            )

    if parsed.path == "/api/analyze/cancel":
        return self._send_json(
```

```
        200,
        self.state.cancel_analysis()
    )
```

Здесь видно несколько вариантов ответа. Отсутствующий project_path считается ошибкой клиента и получает статус 400. Отмена анализа возвращается как отдельное состояние. Ошибка выполнения анализа получает 422 и текст причины. Благодаря этому frontend может отличить разные ситуации и показать соответствующее сообщение.

3.6. Получение проекции графа

Листинг 6:

```
if parsed.path ==
        "/api/graph/projection":
    project_path = str(
        body.get("project_path")
        or self.state.project_path
        or self.state.default_project
        or ""
    ).strip()

    if not project_path:
        return self._send_json(
            400,
            {
                "status": "error",
                "error":
                    "project_path is required"
            }
        )

    try:
        return self._send_json(
            200,
            _graph_projection(
```

```
            project_path,
            body
        )
    )
except (
    ValueError,
    OSError,
    TypeError
) as exc:
    return self._send_json(
        422,
        {
            "status": "error",
            "error": str(exc)
        }
    )
```

Этот endpoint получает параметры фильтрации и возвращает ограниченную проекцию большого графа. Клиент может запросить уровень представления, фильтры и максимальное число узлов и рёбер.

3.7. Состояние анализа

В local_api_state.py хранится текущее состояние backend.

Листинг 7:

```
def snapshot(
    self,
    include_graph: bool = True
) -> dict[str, Any]:

    with self.lock:
        analysis = self.analysis or {}

        result = {
            "status":
```

```
            "error"
            if self.last_error
            else (
                "ready"
                if self.analysis
                else "idle"
            ),
        "has_analysis":
            bool(self.analysis),
        "project_path":
            self.project_path,
        "analyzed_at":
            self.analyzed_at,
        "error":
            self.last_error,
        "progress":
            self.progress,
        "analysis": {
            key: value
            for key, value
            in analysis.items()
            if key != "graph"
        },
    }

    if include_graph:
        result["graph"] =
            analysis.get("graph")

    return result
```

Использование RLock позволяет синхронизировать доступ к состоянию, поскольку анализ и HTTP-обработчики могут выполняться параллельно. Метод snapshot формирует безопасную копию текущего состояния для ответа API.

3.8. Выполнение анализа и обновление прогресса

Листинг 8:

```
def analyze(
    self,
    project_path: str
) -> dict[str, Any]:

    path = Path(
        project_path
    ).expanduser().resolve()

    if not path.exists()        or not path.is_dir():
        raise FileNotFoundError(
            "Project directory does not exist"
        )

    with self.lock:
        if self.analysis_running:
            raise RuntimeError(
                "analysis already running"
            )

        token = CancellationToken()
        self.cancellation = token
        self.analysis_running = True
        self.last_error = None

        self.progress = {
            "status": "running",
            "current": {
                "stage": "starting",
                "message":
                    "Подготовка локального анализа",
                "processed": 0,
                "total": 0,
                "overall_percent": 0.0,
```

```
            "cancellable": True,
        },
    }

def report_progress(event):
    with self.lock:
        self.progress = {
            "status": "running",
            "current": event
        }
```

Backend хранит флаг analysis_running и запрещает запуск второй копии анализа одновременно. Также создаётся CancellationToken, через который пользователь может запросить отмену. Прогресс хранится в состоянии и читается frontend через /api/progress.

4. Экранные формы с примерами выполнения программы

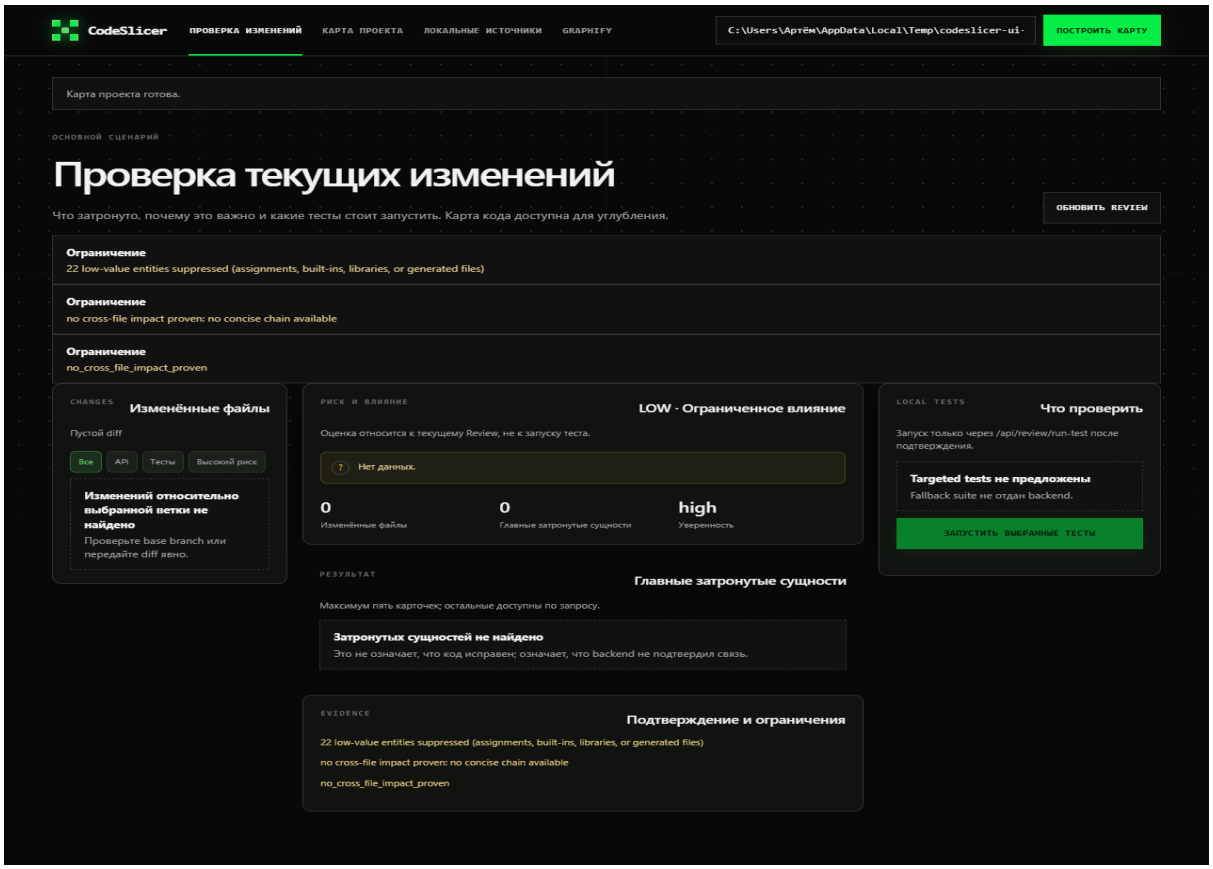


Рисунок 1 — экран взаимодействия с локальным API CodeSlicer.

На экран выводятся данные, которые backend формирует в JSON. Клиент не хранит отдельную статическую копию результатов анализа, а получает состояние от сервера.

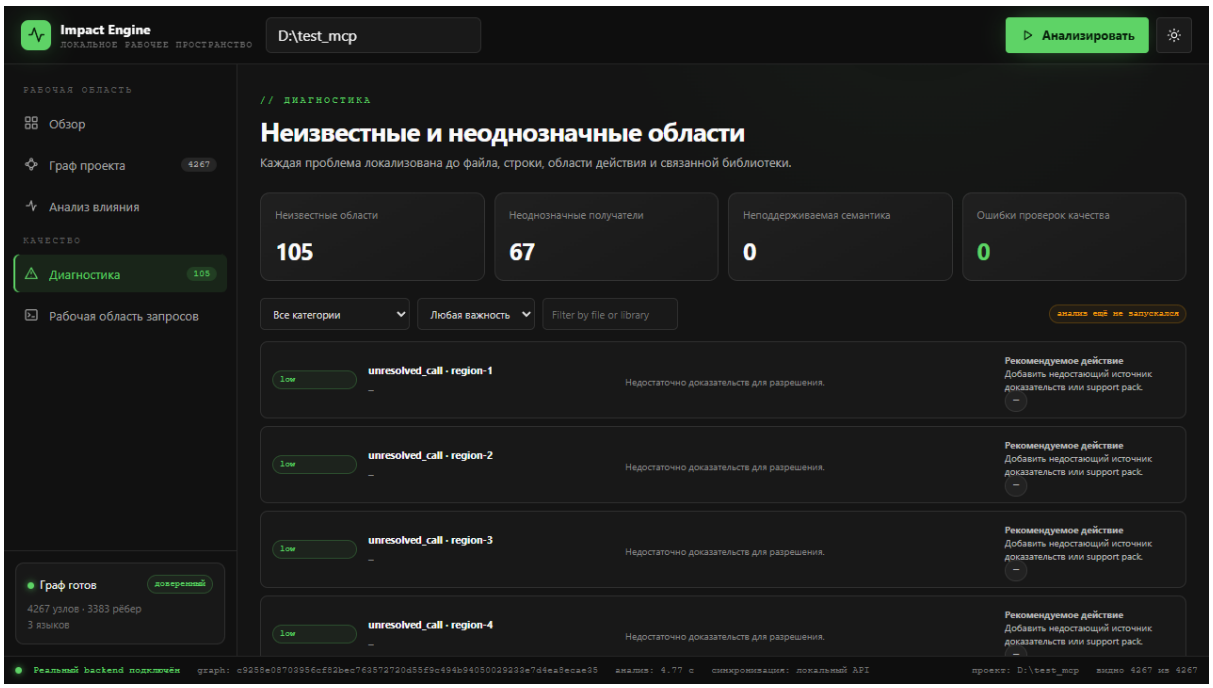


Рисунок 2 — диагностика неизвестных областей.

Диагностическая информация появляется как результат обработки backend. Для пользователя она выглядит как часть интерфейса, однако источником данных являются API-методы и текущее состояние LocalApiState.

Большой граф хранится на серверной стороне, а браузер получает представление, достаточное для конкретного экрана. Это позволяет разделить вычислительную часть и визуализацию.

5. Вывод

В лабораторной работе рассмотрена серверная часть CodeSlicer. На примере реального backend показаны чтение и формирование JSON, обработка GET- и POST-запросов, HTTP-статусы, проверка входных данных, маршрутизация и хранение состояния анализа.

Local API не является отдельной учебной имитацией. Через него действительно запускается анализ проекта, проверяется состояние, запрашивается граф и передаётся прогресс выполнения. При ошибках сервер возвращает структурированные ответы, которые frontend может корректно обработать.

Дополнительно CodeSlicer демонстрирует практический подход local-first: сервер ограничивает обычный доступ loopback-адресами, а основной проект анализируется локально. Это делает серверную часть хорошим примером прикладного HTTP API современного веб-приложения.